

✓ 1. What is API Documentation?

API Documentation explains:

- What endpoints your API exposes
- Request/response formats
- Authentication requirements
- Parameters and validations
- Sample responses

Good documentation helps developers understand and consume your API easily.

✓ 2. What is Swagger / OpenAPI?

Swagger (now called **OpenAPI Specification**) is a standard way to describe REST APIs.

Swagger gives you:

✓ Interactive UI

You can try APIs from the browser.

✓ Auto-generated documentation

Generated from your code.

✓ Request/Response model descriptions

✓ Client SDK generation

Swagger can generate:

- C# client
 - Angular client
 - TypeScript client
 - Java client
-

✓ 3. How Swagger Works in ASP.NET Core

ASP.NET Core uses:

- **Swashbuckle.AspNetCore** → Swagger generator + UI
- **OpenAPI spec** → JSON describing your API

When you run your API, you get:

■ **swagger/v1/swagger.json**

■ **Swagger UI at /swagger**

✓ 4. Step-by-Step Implementation (ASP.NET Core 8 / 7 / 6)

Step 1 — Install the NuGet Package

For .NET 6/7/8:

Install-Package Swashbuckle.AspNetCore

Step 2 — Add Swagger in Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services
```

```
builder.Services.AddControllers();
```

```
// Add Swagger
```

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
```

```
// Enable Swagger in Development/Production
```

```
app.UseSwagger();
```

```
app.UseSwaggerUI();
```

```
app.MapControllers();
```

```
app.Run();
```

Step 3 — Create a Sample API

ProductController.cs

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class ProductController : ControllerBase
```

```
{
```

```
    /// <summary>
```

```
    /// Gets all products.
```

```
    /// </summary>
```

```
    /// <returns>List of products</returns>
```

```
    [HttpGet]
```

```
    public IActionResult GetProducts()
```

```
    {
```

```
        var list = new List<string> { "Laptop", "Mouse", "Keyboard" };
```

```
        return Ok(list);
```

```
    }
```

```

/// <summary>
/// Adds a new product.
/// </summary>
/// <param name="product">Product name</param>
[HttpPost]
public IActionResult AddProduct([FromBody] string product)
{
    return Ok($"{product} added successfully.");
}
}

```

Now open:

👉 <https://localhost:{port}/swagger>

You will see:

- Endpoints
- Parameters
- Schema
- Try it out option

✅ 5. Adding XML Comments for Better Documentation

Step 1 — Enable XML documentation in .csproj

```

<PropertyGroup>
  <GenerateDocumentationFile>true</GenerateDocumentationFile>
</PropertyGroup>

```

Step 2 — Include XML in Swagger (Program.cs)

```
builder.Services.AddSwaggerGen(options =>
{
    var xmlFile = $"{Assembly.GetExecutingAssembly().GetName().Name}.xml";
    var xmlPath = Path.Combine(AppContext.BaseDirectory, xmlFile);
    options.IncludeXmlComments(xmlPath);
});
```

Now Swagger shows:

- Method description
- Parameter description
- Response description

✅ 6. Adding Authorization (JWT) to Swagger

If your API uses **JWT**, Swagger needs to support tokens.

Program.cs

```
builder.Services.AddSwaggerGen(c =>
{
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        In = ParameterLocation.Header,
        Description = "Enter JWT token",
        Name = "Authorization",
        Type = SecuritySchemeType.ApiKey
    });

    c.AddSecurityRequirement(new OpenApiSecurityRequirement {
```

```

{
    new OpenApiSecurityScheme {
        Reference = new OpenApiReference {
            Type = ReferenceType.SecurityScheme,
            Id = "Bearer"
        }
    },
    new string[] {}
});
});

```

Swagger UI now displays a **"Authorize"** button on the top.

You can paste:

Bearer YOUR_JWT_TOKEN

7. Adding Response Types & Status Codes

```

[HttpGet("{id}")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public IActionResult GetProduct(int id)
{
    if (id == 1)
        return Ok("Laptop");

    return NotFound("Product not found");
}

```

Swagger UI now shows **200, 404** with descriptions.

✅ 8. OpenAPI Example Models

```
public class OrderRequest
{
    /// <example>101</example>
    public int ProductId { get; set; }

    /// <example>2</example>
    public int Quantity { get; set; }
}
```

Swagger UI shows example JSON automatically.

✅ 9. Add Example Responses

```
[HttpPost("place")]
[ProducesResponseType(typeof(OrderResponse), 200)]
public IActionResult PlaceOrder(OrderRequest request)
{
    var response = new OrderResponse
    {
        OrderId = 123,
        Status = "Success"
    };

    return Ok(response);
}
```

}

✅ 10. OpenAPI Specification (swagger.json)

Swagger generates:

 /swagger/v1/swagger.json

This is machine-readable and can be used to generate clients in:

- Angular
 - React
 - TypeScript
 - Java
 - Python
 - C#
-

11. Practical Examples of API Documentation

★ Example 1: Public API for Mobile App

- Login
- Register
- Get Products
- Make Payment
- Track Order

All documented automatically.

★ Example 2: Microservices using Swagger + Ocelot

Each service:

- Product Service
- Order Service

- Payment Service

has its own Swagger.

The gateway aggregates all of them.

★ Example 3: External Integrations

Teams using:

- Python
- Node.js
- PHP

can read your swagger.json and quickly generate clients.

Final Summary

Swagger/OpenAPI helps you:

- ✓ Document APIs automatically
- ✓ Test APIs interactively
- ✓ Generate client SDKs
- ✓ Provide example input/output
- ✓ Explain validation rules
- ✓ Enable authorization testing